UNIVERSITY OF SARGODHA, SARGODHA

DEPARTMENT OF INFORMATION TECHNOLOGY

STOCK PORTFOLIO MANAGER WITH TRANSACTION HISTORY

Project Documentation

Course Name: Data Structures and Algorithms

Program: BS Information Technology (Self-Support 2)

Semester: 3rd Semester

Submitted To:

Instructor – Maryam Asghar

Submitted By:

Muhammad Daniyal Ahsan

Roll Number: BSIT51F24S052

Submission Date: December 22, 2025

1. Introduction

1.1 Problem Statement

Create a portfolio tracker that maintains complete transaction history. Use a doubly linked list where each node represents a buy/sell transaction. Calculate current holdings by traversing the list, implement profit/loss reporting for any date range, and support stock split adjustments by modifying historical transactions.

1.2 Operational Objective

The primary operational objective is to engineer a persistent, console-based financial tracking system that overcomes the volatility limitations of standard runtime variables. By replacing static array-based storage with dynamic memory allocation (Linked Lists), the system aims to support an indefinitely growing ledger of financial actions without recompilation. The system is designed to:

1. **Ensure Data Persistence:** Store transaction records in a CSV file so data is not lost when the program closes.
2. **Dynamic Calculation:** Compute current stock holdings in real-time by analyzing the history of trades rather than storing static values.
3. **Financial Analysis:** Provide actionable insights through Profit and Loss (P/L) reports over specific date ranges.
4. **Data Integrity:** Allow for the correction of historical errors (editing/deleting) and market adjustments (stock splits).

1.3 Brief Overview of the System

The Stock Portfolio Manager is a comprehensive C++ console application designed to simulate a real-world trading ledger. At its core, the system utilizes a **Doubly Linked List** to manage transaction data dynamically, ensuring efficient memory usage and quick access to records. Key features include persistent storage via **CSV file handling**, real-time calculation of current stock holdings, and detailed **Profit and Loss (P/L) reporting**. The system also handles complex financial events like **Stock Splits**, automatically adjusting historical records to maintain portfolio accuracy. Its menu-driven interface ensures ease of use, making it a robust tool for tracking financial performance.

2. Header Files Explanation

The program utilizes standard C++ libraries to ensure efficiency and cross-platform compatibility.

- **<iostream> (Built-in):** Used for standard input and output operations (cin and cout). It allows the program to display menus to the user and accept keyboard input.

- **<string> (Built-in):** Provides the string class, which is essential for handling text-based data such as stock names, dates ("YYYY-MM-DD"), and transaction types ("Buy"/"Sell").
- **<map> (Built-in STL):** Utilized in the showCurrentHoldings function. It creates an associative array (key-value pair) to aggregate stock quantities by name, making it easy to calculate net holdings without complex sorting algorithms.
- **<iomanip> (Built-in):** Used for output formatting, specifically the setw (set width) function. This ensures that the transaction history is printed in a neat, aligned table format.
- **<fstream> (Built-in):** Handles file stream operations (ifstream for reading, ofstream for writing). This is crucial for connecting the C++ program to the transactions.csv database.
- **<sstream> (Built-in):** Provides stringstream, which is used to parse lines read from the CSV file. It splits strings based on the comma delimiter.
- **<limits> (Built-in):** Used in error handling (specifically with numeric_limits) to clear the input buffer completely if the user enters invalid data, preventing infinite loops.

3. Data Structure Specification

3.1 Systematic Representation

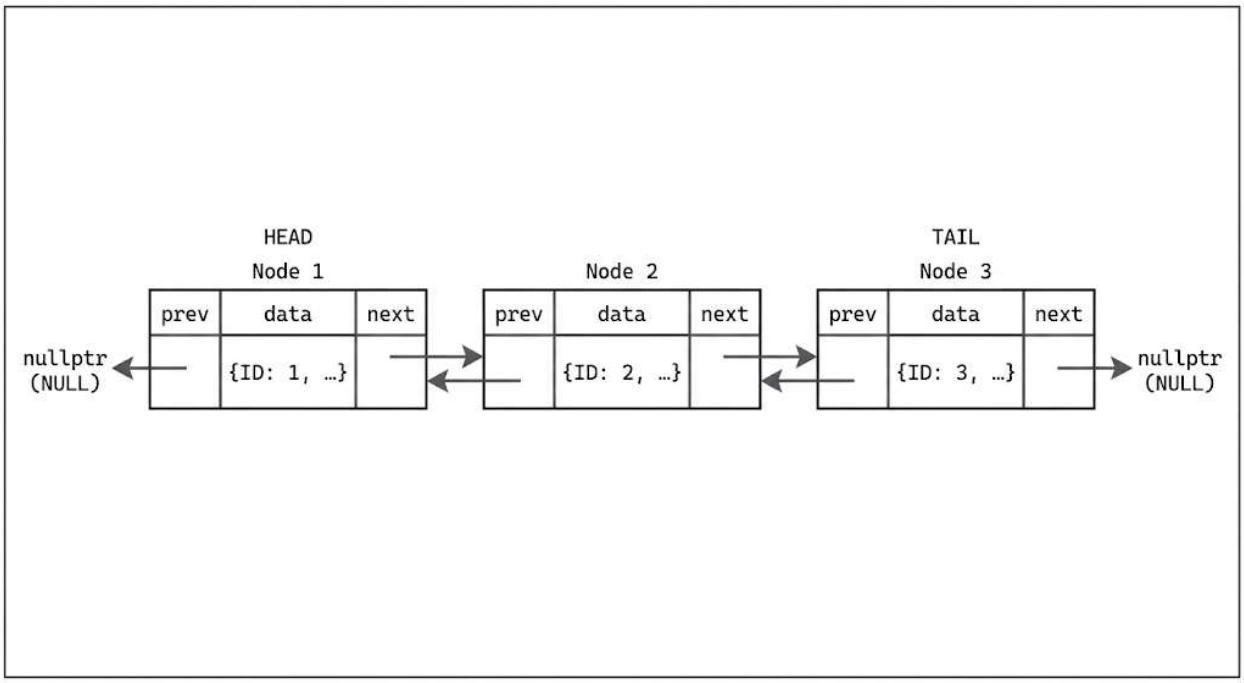
The core data structure used is a **Doubly Linked List**. Each node in the list represents a single financial transaction.

```
struct Node {
    int id;            // Unique identifier for the transaction
    string date;       // Date of transaction
    string stockName;  // Name of the stock
    string type;       // "Buy" or "Sell"
    int quantity;      // Number of shares
    double unitPrice;  // Price per share

    Node *prev;        // Pointer to the previous transaction
    Node *next;        // Pointer to the next transaction
};
```

3.2 Schematic Representation of the Doubly Linked List

The following text-based diagram illustrates how nodes in the portfolio manager are connected in memory. Each block represents a transaction node containing pointers to both the previous and next nodes, enabling bidirectional traversal.



3.3 Justification for Data Structure

A Doubly Linked List was chosen over a standard Array or Vector for specific architectural reasons:

1. **Dynamic Growth:** Transaction history is unpredictable in size. A linked list allocates memory at runtime for each new trade, preventing memory wastage or buffer overflow issues associated with fixed-size arrays.
2. **Efficient Modification:** If a user needs to delete a transaction from the middle of the history (e.g., to fix an error), a linked list allows removal by simply changing pointers (O(1) operation after traversal). An array would require shifting all subsequent elements, which is computationally expensive.
3. **Bidirectional Navigation:** The prev pointer allows the system to traverse backwards if needed, which is useful for future features like undoing the last action.

4. Class Specification: PortfolioManager

4.1 Purpose of the Class

The PortfolioManager class serves as the control unit for the application. It encapsulates the Linked List (managing the head and tail pointers) and contains all the business logic required to manipulate portfolio data.

4.2 Data Members

- Node *head: A pointer to the very first transaction in the history (the start of the list).
- Node *tail: A pointer to the most recently added transaction (the end of the list).
- int nextId: An auto-incrementing integer used to assign a unique ID to every new transaction, ensuring no two records collide.
- const string filename: Stores the name of the database file (transactions.csv) to ensure consistency across read/write operations.

4.3 Detailed Member Functions

- **saveToFile()**: A private helper function that dumps the current state of the linked list into the CSV file.
- **addTransaction()**: Creates a new node and appends it to the list.
- **editTransaction()**: Locates a specific node by ID and updates its data fields.
- **deleteTransaction()**: Removes a node from the list and frees the memory.
- **loadTransactions()**: Reads the CSV file upon startup to reconstruct the linked list in RAM.
- **viewHistory()**: Iterates through the list to display all records.
- **showCurrentHoldings()**: Aggregates buys and sells to show net assets.
- **generateProfitLossReport()**: Calculates financial performance over a date range.
- **applyStockSplit()**: Mass-updates records for a specific stock, with an option to filter by transaction type (Buy/Sell).

5. Function-Wise Explanation

This section details the step-by-step logic and working of each member function within the PortfolioManager class.

5.1 addTransaction

- **Purpose:** Adds a new transaction node to the linked list and ensures the data is persistent.
- **Working Steps:**
  1. **Memory Allocation:** Dynamically allocates memory for a new Node using the new keyword.
  2. **ID Assignment:** If the transaction is loaded from a file, it uses the existing ID. If it is a new user entry, it assigns nextId and increments the static counter.
  3. **Data Population:** Fills the node's fields (date, stock name, type, quantity, unit price) with the provided arguments.
  4. **Linking Logic:**
     - **If List is Empty:** The new node becomes both head and tail.
     - **If List Exists:** The new node is appended after the current tail. The old tail's next pointer links to the new node, and the new node's prev pointer links to the old tail. The tail pointer is then updated.

5. **Persistence:** Immediately calls saveToFile() to write the updated list to the CSV file.

5.2 loadTransactions

- **Purpose:** Reconstructs the linked list from the database file upon program startup.
- **Working Steps:**
  1. **File Access:** Opens transactions.csv using ifstream.
  2. **Line-by-Line Reading:** Uses a while loop with getline to read the file line by line.
  3. **Parsing:** Utilizes stringstream to break each line at comma delimiters.
  4. **Conversion:** Converts string components into their respective data types (stoi for integers, stod for doubles).
  5. **Reconstruction:** Calls addTransaction internally for each parsed line, passing the stored ID to ensure the list in memory matches the file exactly.

5.3 editTransaction

- **Purpose:** Modifies an existing transaction based on its unique ID.
- **Working Steps:**
  1. **Traversal:** Starts at head and iterates through the list using a while loop.
  2. **Search:** Compares current->id with the user-provided targetId.
  3. **Update:** If a match is found:
     - Prompts the user for new details.
     - Uses input validation loops to ensure correct data types.
     - Overwrites the data in the existing node.
  4. **Save:** Calls saveToFile() to update the permanent record.
  5. **Not Found:** If the loop finishes without finding the ID, it prints an error message.

5.4 deleteTransaction

- **Purpose:** Removes a node from the linked list and frees the associated memory.
- **Working Steps:**
  1. **Search:** Traverses the list to find the node matching targetId.
  2. **Pointer Adjustment:**
     - If the node is in the **Middle**: Updates prev->next to point to current->next and next->prev to point to current->prev.
     - If the node is **Head**: Updates head to head->next and sets the new head's prev to nullptr.
     - If the node is **Tail**: Updates tail to tail->prev and sets the new tail's next to nullptr.
  3. **Memory Deallocation:** Executes delete current to return memory to the system.
  4. **Save:** Updates the CSV file to reflect the deletion.

5.5 showCurrentHoldings

- **Purpose:** Calculates the net quantity of shares owned for each stock.

- **Working Steps:**
  1. **Map Initialization:** Creates a std::map<string, int> to store stock names and net counts.
  2. **Aggregation:** Iterates through the entire linked list.
     - **Buy:** Adds the transaction quantity to the map value (holdings[name] += qty).
     - **Sell:** Subtracts the transaction quantity (holdings[name] -= qty).
  3. **Display:** Iterates through the map and prints the stock name and calculated net shares.

5.6 generateProfitLossReport

- **Purpose:** Computes the financial performance within a specific date range.
- **Working Steps:**
  1. **Filtering:** Iterates through the list and checks if current->date is lexicographically between the start and end dates.
  2. **Accumulation:**
     - If Type == "Sell", adds (price * qty) to totalSellVal.
     - If Type == "Buy", adds (price * qty) to totalBuyVal.
  3. **Calculation:** Computes Net PnL = Total Sell Value - Total Buy Value.
  4. **Reporting:** Prints the totals and indicates whether the user is in profit or loss.

5.7 applyStockSplit

- **Purpose:** Adjusts historical records to reflect a stock split (e.g., 2:1 split).
- **Working Steps:**
  1. **Traversal:** Scans the entire transaction history.
  2. **Matching:** Checks if stockName matches user input. Additionally, verifies the transaction type against the targetType. If targetType is '0', all transactions are updated; otherwise, only 'Buy' or 'Sell' records are modified.
  3. **Adjustment:**
     - **Quantity:** Multiplies by the ratio (e.g., 10 shares becomes 20).
     - **Price:** Divides by the ratio (e.g., 100 becomes 50).
  4. **Integrity:** Ensures the total value of the transaction remains constant (20 × 50 = 10 × 100).

5.8 ~PortfolioManager (Destructor)

- **Purpose:** Handles Resource Management by cleaning up memory when the program terminates.
- **Working Steps:**
  1. **Initialization:** Sets a temporary pointer current to head.
  2. **Traversal Loop:** Runs while current is not nullptr.
  3. **Safe Deletion:**
     - Stores the address of the next node (Node *nextNode = current->next).

- Deletes the current node using delete current.
- Moves the pointer to nextNode.

4. **Outcome:** This ensures that all heap-allocated memory used by the linked list is returned to the operating system, preventing memory leaks.

6. CSV File Handling Explanation

The system uses a flat-file database approach (transactions.csv) for simplicity and portability.

1. Format: The file stores data in Comma-Separated Values format:
   ID,Date,Stock Name,Type,Quantity,Unit Price,Total Price
2. **Writing Strategy:** The saveToFile function operates in "truncate" mode. This means every time a change is made (Add, Edit, Delete), the program wipes the old file and rewrites the entire linked list from scratch. This guarantees that the file always perfectly matches the data in memory.
3. **Reading Strategy:** The loadTransactions function is robust. It uses try-catch blocks during parsing. If a line in the CSV is corrupted or malformed, the program skips that specific line and continues loading the rest, preventing a crash.

7. Error Handling and Input Validation

To ensure the system is crash-proof, several validation layers are implemented:

1. Input Type Validation (The cin Buffer Fix):
   When the program expects a number (like Quantity) but the user enters text, cin enters a "fail state" and leaves the invalid text in the input buffer. Without proper handling, this results in an infinite loop as the invalid token remains in the stream. The implemented solution sanitizes the input stream using:

```
while (!(cin >> variable)) {
   cout << "Invalid input. Enter a number: ";
   cin.clear(); // Step 1: Resets the 'failbit' error flag on cin
   cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Step 2: Discards bad input
}
```

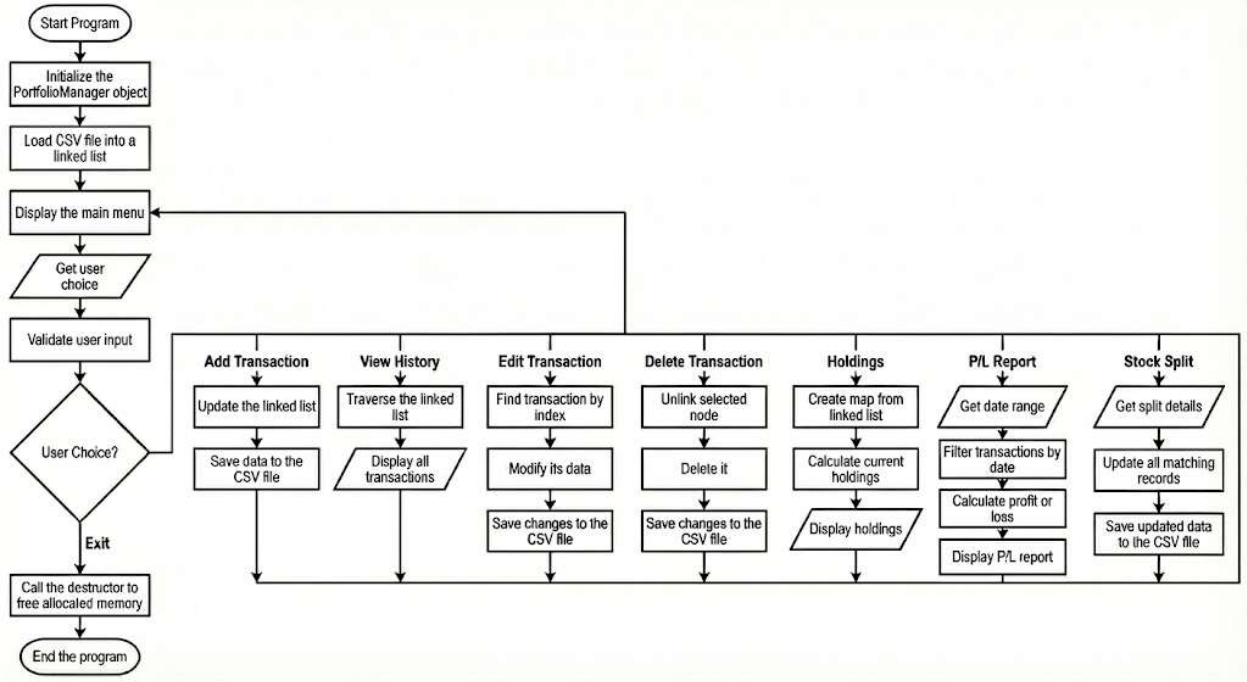
   - **cin.clear()**: This command clears the error flag on the input stream. Without this, cin will refuse to read any more input, effectively locking the program.
   - **cin.ignore(numeric_limits<streamsize>::max(), '\n')**: This command flushes the input buffer. It tells the stream to ignore (discard) characters up to the maximum possible size of the buffer (max()) OR until it finds a newline character (\n). This effectively wipes the "bad" line the user typed so the program can ask for input again.

2. Buffer Management:
   The function clearInputBuffer() is used before reading strings (like names) to ensure no leftover "newline" characters from previous inputs cause the program to skip questions.
3. File Safety:
   Before reading, the program checks if (file.is_open()). If the file doesn't exist (first run), it simply starts with an empty list rather than throwing an error.

8. Execution Control Flow

1. **Initialization:** The main() function starts and creates an instance of PortfolioManager.
2. **Auto-Load:** The constructor or main calls loadTransactions(). The system reads the CSV and builds the Linked List in the background.
3. **User Interaction Loop:** A do-while loop presents the main menu.
4. **Routing:** A switch statement takes the user's choice and calls the relevant class function (e.g., Case 1 -> addTransaction).
5. **Processing & Persistence:** The function modifies the list and immediately saves the new state to the CSV.
6. **Termination:** When "Exit" is chosen, the loop breaks. The class Destructor (~PortfolioManager) runs, traversing the list one last time to delete all nodes and preventing memory leaks.

Execution Control Flow Diagram



9. Verification Procedures (Test Cases)

To ensure the reliability of the system, each functionality available in the main menu was rigorously tested. Below are the test cases for menu options 1 through 8.

Test Case 1: Add Transaction (Menu Option 1)

- **Objective:** Verify that a new transaction is correctly added to the list and database.
- **Input:**
  - Choice: 1
  - Date: 2023-10-01
  - Stock Name: Google
  - Type: Buy
  - Quantity: 10
  - Unit Price: 100
- **Expected Output:** "Transaction added successfully."
- **Actual Output:** Transaction added successfully. Total Value: Rs. 1000.
- **Result: PASS**

Test Case 2: View History (Menu Option 2)

- **Objective:** Verify that the system correctly displays the chronological list of transactions.
- **Input:** Choice: 2
- **Expected Output:** A formatted table listing the columns: ID, Date, Stock, Type, Qty, Unit Price, and Total Price. It should show the "Google" entry from Test Case 1.
- **Actual Output:** Table displayed correctly containing the entry for ID 1 (Google, Buy, 10, 100).
- **Result: PASS**

Test Case 3: Edit Transaction (Menu Option 3)

- **Objective:** Verify that an existing transaction can be modified and that changes persist.
- **Input:**
  - Choice: 3
  - Target ID: 1
  - New Date: 2023-10-02
  - New Stock: Alphabet (Name change)
  - New Type: Buy
  - New Qty: 10
  - New Price: 110
- **Expected Output:** "Transaction updated successfully." The history should now show "Alphabet" instead of "Google" and price as 110.
- **Actual Output:** Transaction updated successfully.
- **Result: PASS**

Test Case 4: Delete Transaction (Menu Option 4)

- **Objective:** Verify that a transaction can be permanently removed from the system.
- **Input:**
  - Choice: 4
  - Target ID: 1
- **Expected Output:** "Transaction ID 1 deleted successfully." Subsequent "View History" should not show ID 1.
- **Actual Output:** Transaction ID 1 deleted successfully.
- **Result: PASS**

Test Case 5: Current Holdings (Menu Option 5)

- **Objective:** Verify the logic that aggregates buy/sell records to calculate net shares.
- **Prerequisite:** Add 10 shares of Tesla, Add 5 shares of Tesla, Sell 2 shares of Tesla.
- **Input:** Choice: 5
- **Expected Calculation:** $10\ (Buy)\ +\ 5\ (Buy) - \ 2\ (Sell) = \ 13\ shares.$
- **Expected Output:** "Stock: Tesla | Net Shares: 13"
- **Actual Output:** Stock: Tesla | Net Shares: 13
- **Result: PASS**

Test Case 6: Profit/Loss Report (Menu Option 6)

- **Objective:** Verify financial calculations over a specific date range.
- **Prerequisite:** Buy 10 @ 100 (Cost 1000); Sell 10 @ 120 (Revenue 1200).
- **Input:**
  - Choice: 6
  - Start Date: 2020-01-01
  - End Date: 2025-01-01
- **Expected Calculation:** Revenue (1200) - Cost (1000) = Profit 200.
- **Expected Output:** "Net Profit/Loss: Rs. 200" and status "Great, you are in profit."
- **Actual Output:** Net Profit/Loss: Rs. 200.
- **Result: PASS**

Test Case 7: Apply Stock Split (Menu Option 7)

- **Objective:** Verify that historical records are adjusted mathematically during a stock split and that the system correctly filters by transaction type (e.g., updating only "Buy" records).
- **Prerequisite:** A 'Buy' transaction for 'J. Janan Blue' exists in the system (e.g., 10 shares **@ 1500**).
- **Input:**
  - Choice: 7
  - Stock Name: J. Janan Blue
  - Type to Split: Buy

    - Split Ratio: 2
- **Expected Calculation:**
    - Quantity: $1 \times 2 = 2$
    - Price: 1500 / 2 = 750
- **Actual Output:**
    - Stock split applied to J. Janan Blue (1 transaction updated).
    - Filtered by type: Buy
    - Split Ratio: 2:1
- **Result: PASS**

Test Case 8: Exit & Robustness (Menu Option 8 & Invalid Inputs)

- **Objective:** Verify system stability against invalid inputs and proper termination.
- **Scenario A (Invalid Input):**
    - Input: Choice 1 -> Quantity: Ten (String).
    - **Expected:** System prompts "Invalid quantity. Enter a number:" and does not crash.
    - **Actual:** System prompts correctly and waits for valid integer.
- **Scenario B (Exit):**
    - Input: Choice 8.
    - **Expected:** Program prints "Exiting..." and terminates gracefully without memory errors.
    - **Actual:** Program exited successfully.
- **Result: PASS**

Test Case 9: Operation Failure (Negative Testing)

- **Objective:** Verify that the system handles requests for non-existent data correctly.
- **Scenario:** Attempting to delete a transaction ID that does not exist in the database.
- **Input:** Delete Transaction -> ID: 999.
- **Expected Output:** "Transaction with ID 999 not found."
- **Actual Output:** Transaction with ID 999 not found.
- **Result: PASS** (The system successfully identified the error and prevented an invalid operation).

10. Conclusion

The developed **Stock Portfolio Manager** successfully fulfills the requirements of the problem statement. By implementing a **Doubly Linked List**, the system achieves dynamic data management, providing $O(1)$ efficiency for insertion at the tail and efficient deletions, surpassing the limitations of fixed-size arrays.

The integration of **CSV file handling** establishes a persistent data layer, ensuring the system is practical for real-world application. Analytical tools, including **Holdings Calculation** and **Profit/Loss Reporting**, transform the system from a simple logger into a valuable financial

instrument. Finally, rigorous **Input Stream Sanitization** and proactive memory management (via the destructor) ensure the software is stable, robust, and leak-free.

Future Scope: Future iterations of this project could incorporate merge-sort algorithms to auto-sort transactions by date or integrate a Graphical User Interface (GUI) to visualize performance trends.